

Data Models, Data Preparation

*Instructor: Aditya Parameswaran**Covers Lecture(s): 9 – 10*

I Identifying Data Models

For each of the following data models, determine whether it is a matrix, relation, or dataframe.

1. What is this data model?

	gender	name	age	id
1	F	Ariel	32	12345
2	M	Devin	24	23456
3	F	Mary	28	34567
4	M	Eric	30	45678

Dataframe because both rows and columns have names/labels.

2. What is this data model?

34	12	NaN	45
234	NaN	NaN	657
45	90	91	23
8	1	NaN	0

Matrix because there are no column or row labels.

3. What is this data model?

City	State	Country	Zip
New York	NY	USA	10012
Berkeley	CA	USA	94702
Berkeley	CA	USA	94705

Relation because there are only column labels, no row labels.

4. What is this data model?

Index	Last Name	School	Class
1	Tassens	UC Berkeley	2012
2	Lee	UCSF	2014
3	Smith	UC Berkeley	2015
4	Sanchez	UCSF	2013

Relation because there are only column labels, no row labels. Note that the Index column is not a row label.

II Data Model Concepts

For questions 5-8, determine whether each of the following statements is true or false. Explain.

5. All matrices are tensors and all tensors are matrices.

True and false. Matrices are 2-dimensional tensors, but tensors need not be confined to 2 dimensions. For example, a vector is a 1-dimensional tensor, and there could be 3- or 4-dimensional tensors, too.

6. Matrices with lots of zeros are said to be sparse. Dense matrices contain very few zeros.

True.

7. Tensors are the general term for N-dimensional matrices.

True. A tensor of rank 1 is called a vector, a tensor of rank 2 is called a matrix, and higher-rank tensors represent multi-dimensional arrays.

8. A tensor of shape $4 \times 5 \times 6$ has tensor rank 4.

False. This is a 3-dimensional array so the tensor rank is 3.

9. Express the following relation as a matrix, and the matrix as a relation. True or False: All relations can always be expressed as matrices and vice versa. Explain

16.2	11.6	NaN
19.4	NaN	14.2

Student	Course ID	Score
Ariel	101	79.2
Ariel	102	92.1
Devin	101	85.0
Mary	103	90.0

RowID	ColID	Val
0	0	16.2
0	1	11.6
1	0	19.4
1	2	14.2

Ariel	Devin	Mary	Course ID	Score
1	0	0	101	79.2
1	0	0	102	92.1
0	1	0	101	85.0
0	0	1	103	90.0

False and true. Relations that contain only numerical values can be easily converted to matrices. If a relation contains categorical variables, we can convert it into a matrix using one-hot encoding (as shown above). However, if a relation contains arbitrary string fields, it would be prohibitively expensive to convert it into a matrix using one-hot encoding. The converse is true; we can always convert a matrix into a relation by adding appropriate schema definitions and row indices.

10. Consider the following dataframe.

	Student	Course ID	Score
0	Ariel	101	79.2
1	Ariel	102	92.1
2	Devin	101	85.0
3	Mary	103	90.0

- (a) Transpose the dataframe. Assuming the original types are, in order: **String**, **Integer**, and **Float**. What are the dtypes of each transposed column?

	0	1	2	3
Student	Ariel	Ariel	Devin	Mary
CourseID	101	102	101	103
Score	79.2	92.1	85.0	90.0

All columns are type **object**. Dataframe libraries such as Pandas enforce column type homogeneity. The values in the row each had different types, after transposition, they become members of the same column and are coerced into a single type.

- (b) If we transpose the dataframe again, would we get back the original dataframe? Explain.

In theory, we should get back the original dataframe, but in practice, Pandas may change the types during the first transpose (i.e. into **objects**), and when we transpose back, we lose the original type information, even though the data looks the same.

For example, after the double transpose, the 'Score' column would contain string objects like "79.2" and arithmetic operations can fail

III Pivoting and Unpivoting

For this section, we recommend you open DataHub so you can try running these queries yourself in PostgreSQL. For your convenience, we also list the questions here.

11. *Pivoting*. Write a Pandas expression to reshape the following table such that its columns are the 3 types of metals in the CType column, and whose values are taken from the USD column. Also write the resulting table. (Hint: Use Item as the index.)

	Item	CType	USD	EUR
0	Item0	Gold	\$1	€1
1	Item0	Bronze	\$2	€2
2	Item1	Gold	\$3	€3
3	Item1	Silver	\$4	€4

```
df.pivot(index="Item", columns="CType", values="USD")
```

Result:

	Item	Bronze	Gold	Silver
0	Item0	\$2	\$1	NaN
1	Item1	NaN	\$3	\$4

12. *Unpivoting*. Write a Pandas expression to reshape the following table from a *wide* shape into a *long* shape, using the student and school columns as identifiers, and the rest of the subjects as variables. Also write the resulting table. Hint: use `df.melt()`.

	student	school	english	math	physics
0	Andy	Z	90	60	80
1	Bernie	Y	100	5	4
2	Cindy	Z	60	99	100
3	Deb	Y	150	4	5

```
df.melt(id_vars=["student", "school"], value_vars=["english", "math", "physics"],  
var_name="Subject", value_name="Score")
```

Result:

student	school	Subject	Score
Andy	Z	English	90
Bernie	Y	English	100
Cindy	Z	English	60
Deb	Y	English	150
Andy	Z	Math	60
Bernie	Y	Math	5
Cindy	Z	Math	99
Deb	Y	Math	4
Andy	Z	Physics	80
Bernie	Y	Physics	4
Cindy	Z	Physics	100
Deb	Y	Physics	5

IV Storage Size Calculation

Consider the following schema.

```
CREATE TABLE orders(  
  student_id INTEGER, student_name VARCHAR(30), order_id INTEGER,  
  order_info VARCHAR(50), price DOUBLE,  
  FOREIGN KEY (student_id) REFERENCES students (student_id)  
);
```

13. Suppose there are 300 orders in the `orders` table. Compute the total storage cost required to store all the columns in bytes. For reference:

- CHARs are 1 byte per character.
- INTEGERS are 4 bytes.
- DOUBLES are 8 bytes.

Total Bytes per Row = $4 + 30 + 4 + 50 + 8 = 96$ bytes

Total Storage Cost = $300 \text{ rows} \times 96 \text{ bytes per row} = 28,800$ bytes

V Minimum Description Length

Consider the relation below for this section.

Name	Species	Friendly?	NumItems
<i>TomNook</i>	<i>Raccoon</i>	1	5.0
<i>Timmy</i>	<i>Raccoon</i>	1	10
<i>Tommy</i>	<i>Raccoon</i>	1	20.0
<i>Blathers</i>	<i>Owl</i>	0	110.0
<i>Wisp</i>	<i>Ghost</i>	0	<i>Null</i>
<i>Pete</i>	<i>Pelican</i>	1	67.0
<i>Cyrano</i>	<i>Anteater</i>	0	23
<i>Mabel</i>	<i>Hedgehog</i>	1	14

14. Identify the data type for each column.

Name: String;

Species: String;

Friendly?: Boolean;

NumItems: Float.

We choose the data type that takes up the least space while still accommodating all possible values in the column.

15. We will use MDL to determine the optimal type for the `NumItems` column. Assume floats are 32 bits, integers are 16 bits (although integers can also be 32 bits, let's assume we are using `smallint` here), booleans are 1 bit, and a character is 8 bits.

(a) We first consider integer type. What would be the result of the sum in the MDL equation?

$$32 + 16 + 32 + 32 + 0 + 32 + 16 + 16 = 176$$

The exceptions here are the float and Null values on first, third, fourth, fifth, and sixth rows. To explicitly encode a float number we need 32 bits. Other rows can be encoded into integers and only need 16 bits. Null requires 0 bit.

(b) Now we consider float. What would be the result of the sum in the MDL equation?

$$32 + 32 + 32 + 32 + 0 + 32 + 32 + 32 = 32 \times 7 = 224$$

All the records except Null in this column can be encoded into 32-bit float and Null requires 0 bit.

(c) Now consider the string type. What would be the result of the sum in the MDL equation?

$$(3 + 2 + 4 + 5 + 4 + 4 + 2 + 2) \times 8 = 208$$

Each record will take $8n$ bits where n is the string length in characters.

Note: Null is encoded as a 4-character string, "Null". Whereas in numeric units it cannot be represented and costs 0 bits.

(d) Which type would MDL choose based on the calculations in the previous parts?

MDL chooses to minimize the sum value calculated so it will choose the integer type for `NumItems` column.

16. Suppose Tom Nook's `NumItems` is the string `five` instead of the number 5.0.

(a) How would this affect the MDL sum for integer type?

$$32 + 16 + 32 + 32 + 0 + 32 + 16 + 16 = 176 \text{ Incidentally, the sum stays the same, because "five" takes } 8 \times 4 = 32 \text{ bits, same as a floating point number.}$$

(b) If we do use the integer column type, how would you coerce the value `five` into an integer?

In most database systems, coercing a non-numeric string like 'five' to an integer will result in an error or NULL, because 'five' does not directly represent a numeric value. In other cases, custom logic would need to be implemented to manually convert specific strings (e.g., 'five' to 5).

VI Data Preparation [Optional]

Consider the following table:

Year	College	Student Name
2019	L&S	Lakshya
2019	CNR	Shadaj
2020	Haas	Cathy
2020	COE	Evelyn
2020	COC	Luke
2021	CNR	Shreya
2022	L&S	Audrey
2022	COE	Alan

17. Describe how we transform a relation into a matrix and vice versa using Pandas. What functions do we use?

To turn a matrix into a relation, we use the `melt()` function in Pandas to unpivot the matrix.
To turn a relation into a matrix, we use the `pivot()` function in Pandas.

18. Transform the above relation to a matrix using college as columns and year as rows.

2019	Shadaj	NaN	NaN	NaN	Lakshya
2020	NaN	Cathy	Evelyn	Luke	NaN
2021	Shreya	NaN	NaN	NaN	NaN
2022	NaN	NaN	Alan	NaN	Audrey

19. Is the matrix sparse or dense?

Sparse because many values are NaN.

20. Transform this matrix to a sparse representation, expressed as a relation: $\begin{bmatrix} 1 & 2 & 3 & NaN \\ NaN & NaN & 7 & 8 \end{bmatrix}$

RowID	ColID	value
0	0	1
0	1	2
0	2	3
1	2	7
1	3	8